

Eactive Technologies – Software Engineer Trainee Assignment

Professional Project Documentation

Objective

This project demonstrates proficiency in Python (Flask or Django), MySQL, and Git by building a small web application that allows user management and database interaction.

Environment Setup

Install Python 3.x on your machine.

Install Flask using: `pip install flask`

Install MySQL and create a local database named 'users'.

Task 1: Flask API Development

Project Structure:

flask_app/

app.py

Templates/

Users.html

new_user.html

Static/

Example Flask Code (app.py):

```
from flask import Flask, render_template, request, redirect, url_for, abort
import mysql.connector

app = Flask(__name__)
```

```
# Database configuration

db_config = {

    'host': 'localhost',

    'user': 'root',    # your MySQL username

    'password': 'Som12345',    # your MySQL password

    'database': 'userdb'

}
```

```
def get_db_connection():

    conn = mysql.connector.connect(**db_config)

    return conn
```

```
# Route 1: Hello World
```

```
@app.route('/hello')
```

```
def hello():

    return "Hello Anshul!"
```

```
# Route 2: Show all users
```

```
@app.route('/users')
```

```
def users():

    conn = get_db_connection()

    cursor = conn.cursor(dictionary=True)

    cursor.execute("SELECT * FROM users")

    users = cursor.fetchall()

    cursor.close()
```

```
conn.close()

return render_template('users.html', users=users)
```

Route 3: Add new user form

```
@app.route('/new_user', methods=['GET', 'POST'])
```

```
def new_user():
```

```
    if request.method == 'POST':
```

```
        name = request.form['name']
```

```
        email = request.form['email']
```

```
        conn = get_db_connection()
```

```
        cursor = conn.cursor()
```

```
        cursor.execute("INSERT INTO users (name, email) VALUES (%s, %s)", (name, email))
```

```
        conn.commit()
```

```
        cursor.close()
```

```
        conn.close()
```

```
        return redirect(url_for('users'))
```

```
    return render_template('new_user.html')
```

Route 4: User detail by ID

```
@app.route('/users/<int:id>')
```

```
def user_detail(id):
```

```
    conn = get_db_connection()
```

```
    cursor = conn.cursor(dictionary=True)
```

```
    cursor.execute("SELECT * FROM users WHERE id = %s", (id,))
```

```

user = cursor.fetchone()

cursor.close()

conn.close()

if not user:

    abort(404, description="User not found")

return render_template('user_detail.html', user=user)

# Error Handling

@app.errorhandler(404)

def not_found(error):

    return f"<h1>404 - {error.description}</h1>", 404

if __name__ == '__main__':

    app.run(debug=True)

```

Task 2: Database Interaction

Database Schema:

Database: users

Table: users (id, name, email, role)

SQL Queries:

CREATE TABLE users (

```
id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100),  
email VARCHAR(100), role VARCHAR(50)  
);
```

```
INSERT INTO users (name, email, role) VALUES ('Alice', 'alice@example.com', 'Developer'),  
('Bob', 'bob@example.com', 'Designer'), ('Charlie', 'charlie@example.com', 'Tester');
```

```
SELECT * FROM users;
```

```
SELECT * FROM users WHERE id = 1;
```

Task 3: Version Control with Git

```
git init  
git checkout -b assignment  
git add .  
git commit -m "Initial commit with Flask and MySQL setup"  
git remote add origin <your_repo_url>  
git push origin assignment
```

Task 4: Documentation

This README file serves as project documentation and setup instructions. How to Run the Application:

```
python app.py http://127.0.0.1:5000/
```

Git Workflow Summary

Initialize repo: `git init`

Create branch: `git checkout -b assignment` Commit changes: `git commit -m "message"`

Push branch: `git push origin assignment` Pull Request: via GitHub/GitLab

Contact

Number - 8349922001